Reviewer Pack — Minimal Order of Quasigroup Representations and Circuit Mono…

Entry a28922d3d1fb · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-02 03:50:38 UTC

Minimal Order of Quasigroup Representations and Circuit Monotone Width Correlation

Entry ID: `a28922d3d1fb` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-02 03:50:38 UTC

1. Conjecture

Field A (mathematical branch): Quasi-group Theory **Field B** (complexity object): Boolean Circuits

Statement:

> The minimal order of quasigroup representations for a given boolean circuit is linearly correlated with its circuit monotone width, such that the minimal order O(n) = Θ(MonW(circuit)) where MonW(circuit) represents the circuit's monotone width and n is the number of variables in the circuit.

Rationale (proposer's reasoning):

> Quasi-group theory provides a combinatorial framework for studying symmetries. By associating quasigroup representations with boolean circuits, we can explore new ways to understand the structure of circuits that may not be evident from traditional complexity measures like monotone width. If there is a significant correlation, it suggests a novel connection between group-theoretic properties and computational complexity.

Taxonomy category: `Quasi-group_Boolean_Circuits` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `72ec917dfed4dea8`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For a boolean circuit with n variables, if the minimal order of its quasigroup representation (O(n)) is within a factor of 2 from the monotone width (MonW(circuit)) and all seeds produce a correlation coefficient greater than 0.7, this supports the conjecture.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def evaluate_circuit(circuit, inputs):
        for gate in circuit:
            if gate[0] == 'AND':
                inputs.append(inputs.pop() and inputs.pop())
            elif gate[0] == 'OR':
                inputs.append(inputs.pop() or inputs.pop())
            elif gate[0] == 'NOT':
                inputs[-1] = not inputs[-1]
        return inputs[0]

    def truth_table(circuit, n):
        table = []
        for i in range(2**n):
            binary_input = [bool((i >> j) & 1) for j in range(n)]
            result = evaluate_circuit(circuit, binary_input[:])
            table.append((binary_input, result))
        return table

    def quasigroup_representation(table):
        n = len(table[0][0])
        elements = set()
        for inputs, _ in table:
            for bit in inputs + [table[0][1]]:
                elements.add(bit)
        elements = sorted(elements)

        qg = {}
        for i, a in enumerate(elements):
            for j, b in enumerate(elements):
                qg[(a, b)] = elements[(i * len(elements) + j) % len(elements)]
```

```
    return qg

def minimal_order(qg):
    n = len(qg)
    order = 1
    while True:
        found = False
        for a in range(n):
            for b in range(n):
                if qg[(qg[(a, b)], qg[(b, a)])] != qg[(a, b)]:
                    found = True
                    break
            if found:
                break
        if not found:
            return order
        order += 1

def monotone_width(circuit):
    n = len(circuit)
    width = [0] * (n + 1)
    for gate in circuit:
        if gate[0] == 'AND' or gate[0] == 'OR':
            width[gate[2]] = max(width[gate[1]], width[gate[3]]) + 1
        elif gate[0] == 'NOT':
            width[gate[2]] = width[gate[1]]
    return max(width)

def correlation(x, y):
    n = len(x)
    mean_x = sum(x) / n
    mean_y = sum(y) / n
    cov = sum((x[i] - mean_x) * (y[i] - mean_y) for i in range(n)) / n
    std_x = math.sqrt(sum((x[i] - mean_x) ** 2 for i in range(n)) / n)
    std_y = math.sqrt(sum((y[i] - mean_y) ** 2 for i in range(n)) / n)
    return cov / (std_x * std_y)

results = []
n_values = [5, 10, 15, 20, 30, 40]
for n in n_values:
    for _ in range(5):
        circuit = []
        for i in range(n - 1):
            gate_type = random.choice(['AND', 'OR'])
            inputs = [random.randint(0, n-1) for _ in range(gate_type == 'OR')]
            circuit.append((gate_type, *inputs))
        table = truth_table(circuit, n)
        qg = quasigroup_representation(table)
        order = minimal_order(qg)
        mon_width = monotone_width(circuit)
        results.append((order, mon_width))

if len(results) < 30:
    return {
        "metric_name": "correlation",
        "metric_value": None,
```

```
            "instances_tested": len(results),
            "n_max": max(n_values),
            "conjecture_holds": False,
            "counterexample": "insufficient_data"
        }

    orders, mon_widths = zip(*results)
    corr = correlation(orders, mon_widths)

    return {
        "metric_name": "correlation",
        "metric_value": corr,
        "instances_tested": len(results),
        "n_max": max(n_values),
        "conjecture_holds": corr > 0.7,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43,
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_corr = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(res
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_corr} std=0.0 support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results) and support_fraction >= 0.8:
        print(f"RESULT: FALSIFIED counterexample=\"correlation_too_low\" first_failing_seed={seeds
    else:
        print(f"RESULT: INCONCLUSIVE support_fraction={support_fraction}")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test timed out before producing data, which means it did not meet the pre-registered support condition for the conjecture. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	22328
2	preregistration	ollama_remote	glm4:latest	0	0	22978
3	novelty	ollama_remote	glm4:latest	0	0	15734
4	novelty	ollama_remote	glm4:latest	0	0	23302
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10719
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19937
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12278
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15487
9	judge	ollama_remote	glm4:latest	0	0	8400

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 151163 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/a28922d3d1fb.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/a28922d3d1fb.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/a28922d3d1fb.tar.gz` (if generated)